Tangle Network

The Operating Layer for Autonomous Work

Protocol Specification v1.0

Tangle Foundation

June 2026

Abstract

Tangle Network is a decentralized protocol for coordinating computational services with cryptoeconomic accountability. This document describes the protocol's architecture, economic mechanisms, and security model. Developers define service templates (blueprints), operators provide compute with staked collateral, and customers — human or agent — consume services with cryptographic guarantees. The core accounting paths for staking, slashing, and reward distribution are O(1), so gas cost does not grow with participant count. Operators stake assets that can be destroyed if they misbehave, which makes honest service the economically rational path. The network token (TNT) is hard-capped at 100,000,000 TNT, with the full supply minted at genesis; "inflation" rewards are treasury-funded emissions that create no new supply.

Contents

1 Introduction **4**
- 1.1 The Infrastructure Question . . . 4
- 1.2 Concrete Scenarios . . . 4
- 1.3 Design Philosophy . . . 5
  - 1.3.1 Democratized Ownership . . . 5
  - 1.3.2 Permissionless Participation . . . 5
  - 1.3.3 Developer Expressiveness . . . 6
  - 1.3.4 Economic Security . . . 6
  - 1.3.5 Scalable Efficiency . . . 6
  - 1.3.6 Isolation by Default . . . 6
  - 1.3.7 Navigating Tensions . . . 7

2 Blueprints and Services **7**
- 2.1 What Is a Blueprint? . . . 7
- 2.2 What Is a Service Instance? . . . 7
- 2.3 Service Lifecycle . . . 7
- 2.4 Pricing Models . . . 8
- 2.5 Membership Models . . . 8
- 2.6 Master Blueprint Service Manager (MBSM) . . . 8
  - 2.6.1 Architecture . . . 8
  - 2.6.2 Version Management . . . 9
  - 2.6.3 Why This Matters . . . 9

3 Operators and Staking **9**
3.1 Operator Registration . . . 9
3.2 Delegation Modes . . . 9
3.3 Liquid Delegation Vaults . . . 9
3.4 Exposure Selection . . . 10
3.5 O(1) Share-Based Accounting . . . 10

4 The Service Marketplace **10**
4.1 Why Request-for-Quote . . . 10
4.2 The RFQ Flow . . . 11
4.3 Resource Pricing . . . 11
4.4 Security Commitments and Developer Requirements . . . 11
4.5 Quote Security . . . 11
4.6 MEV Considerations . . . 11

5 Job Execution **12**
5.1 Job Submission . . . 12
5.2 Result Submission . . . 12
5.3 Aggregation Modes . . . 12
5.4 Failure Modes . . . 12
5.4.1 Operator Timeout . . . 12
5.4.2 Result Disagreement . . . 13
5.4.3 Invalid Results . . . 13
5.4.4 Service Unavailability . . . 13
5.5 Job Execution Economics . . . 13

6 Fee Distribution and Rewards **13**
6.1 Service Fee Splits . . . 14
6.2 Operator Revenue . . . 14
6.3 Delegator Rewards . . . 14
6.4 Streaming Payments . . . 14
6.5 Developer Rewards . . . 15

7 Inflation and Epoch Distribution **15**
7.1 Token Supply, Genesis Allocation, and Migration . . . 15
7.2 The Pre-Funded Model . . . 15
7.3 Distribution Weights . . . 16
7.4 Operator Scoring . . . 16

8 Slashing and Accountability **16**
8.1 Developer-Defined Conditions . . . 16
8.2 The Dispute Window . . . 16
8.3 Proportional Slashing . . . 17
8.4 Withdrawal Protection . . . 17

9 Governance **17**
9.1 Governance Philosophy . . . 17
9.2 Governance Mechanism . . . 17
9.3 Worked Example: Fee Parameter Change . . . 18
9.4 Attack Resistance . . . 18
9.5 Vote Delegation . . . 18
9.6 Emergency Mechanisms . . . 19

9.7 Governable Parameters . . . 19
9.8 Upgrade Mechanism . . . 19
9.8.1 Facet Router Architecture . . . 19

10 Security Model **20**
10.1 Threat Model . . . 20
10.2 Economic Security Analysis . . . 20
10.2.1 Cost of Corruption . . . 20
10.2.2 Profit from Corruption . . . 20
10.2.3 Security Ratio . . . 21
10.2.4 Worked Example: Trading Agent Service . . . 21
10.2.5 Flash Loan Resistance . . . 21
10.3 Attack Vectors and Mitigations . . . 21
10.4 Formal Security Properties . . . 22
10.5 Security Assumptions . . . 22
10.6 Security Recommendations . . . 23
10.7 Residual Risks . . . 23

11 Comparison to Existing Protocols **23**
11.1 The Restaking Landscape . . . 23
11.2 Architectural Comparison . . . 23
11.3 Key Differentiators . . . 24
11.4 When to Choose Each Protocol . . . 24

12 Beacon Chain Integration **24**

13 Conclusion **25**

A Mathematical Foundations **25**
A.1 Share-Based Accounting . . . 25
A.1.1 Deposit . . . 25
A.1.2 Withdrawal . . . 25
A.1.3 Value Calculation . . . 26
A.2 Accumulated-Per-Share Rewards . . . 26
A.3 Fee Split Calculation . . . 26
A.4 Operator Scoring . . . 26
A.5 Developer Scoring . . . 27
A.6 Proportional Slashing . . . 27
A.7 Security Economics . . . 27

1 Introduction

Increasingly capable AI systems need infrastructure that can scale without losing accountability. Current options fall into two weak categories: centralized cloud providers that can revoke access or change terms, and decentralized compute markets that often lack the coordination needed for production workloads. Centralized systems also concentrate pricing power, operational control, and customer relationships in a few companies.

Tangle fills that gap with a protocol layer for computational services. Developers define service templates. Operators provide compute and stake collateral. Customers request service instances and pay for work. Service fees and treasury-funded emission then flow to participants according to contribution rather than control of distribution.

1.1 The Infrastructure Question

AI agents capable of autonomous action require infrastructure that matches their capabilities:

- An agent managing a portfolio needs compute that cannot be arbitrarily terminated
- An agent processing sensitive data needs isolation guarantees beyond contractual promises
- An agent making consequential decisions needs audit trails that cannot be falsified

Traditional cloud infrastructure provides compute but not hard accountability. Terms can change, access can be revoked, and the provider set is small. Dependence becomes market power. For infrastructure that controls meaningful economic activity, that is a real risk.

Tangle provides an alternative where accountability is cryptographic rather than contractual. Operators stake assets that can be destroyed if they violate service terms. Customers pay for services with cryptographic proof of delivery. The protocol coordinates without controlling, enabling a competitive market of independent operators rather than dependence on a few large providers.

1.2 Concrete Scenarios

Four scenarios illustrate where current infrastructure fails and where cryptoeconomic accountability succeeds.

Scenario 1: The Trading Agent. An institutional investor deploys an AI agent managing $100 million across decentralized exchanges. The agent monitors markets, identifies opportunities, and executes trades.

In the centralized model, the cloud provider observes trading patterns, positions, and strategies. Nothing prevents front-running or information extraction. The investor has no cryptographic guarantee of proper execution, no recourse for information asymmetry, and no audit trail of what actually occurred.

In Tangle, the agent runs on operator infrastructure where execution produces cryptographic attestations. Operators stake assets slashable for information leakage. Multiple operators verify through redundant computation. The investor has on-chain evidence, economic recourse, and operator selection based on reputation and stake.

Scenario 2: The Research Agent. A pharmaceutical company deploys AI agents analyzing clinical trial data worth billions in competitive advantage.

In the decentralized-compute model, providers see processed data. Network operators observe data flows. Disputes require human adjudication by potentially conflicted parties.

In Tangle, blueprints define verification mechanisms: TEEs for isolation, MPC for analysis without revealing inputs, redundant execution with cryptographic comparison. Slashing conditions specify penalties for leakage. Operators are selected based on security practices and stake. Disputes resolve through on-chain mechanisms.

Scenario 3: The Coding Agent. A software company deploys AI agents maintaining codebases representing years of development effort.

In the centralized model, the provider has contractual obligations but no economic penalty for breach. Recourse is litigation, slow, expensive, uncertain. The provider minimizes security spending up to expected litigation costs.

In Tangle, operators stake assets proportional to code value. Breach triggers immediate, automatic slashing. Operators invest in security up to stake value. The company selects operators whose stake matches or exceeds code value.

Scenario 4: The Agent as Customer. An autonomous agent must procure compute for its own work: inference, data analysis, sandboxed execution. It holds a wallet and a budget; no human approves each purchase.

In the centralized model, the agent cannot be a customer. Accounts require identity verification and contractual terms that presume a legal person. The agent's principal must preprovision credentials with a single provider, recreating the dependence the deployment was meant to avoid.

In Tangle, the agent is a customer like any other. It discovers blueprint services, collects EIP-712-signed quotes through the RFQ flow, and hires operators without committee approval. Event-driven pricing charges per job, so spend tracks work exactly. Every job produces the same cryptographic accountability evidence available to any customer, giving the agent's principal an audit trail of what was purchased and what was executed.

1.3 Design Philosophy

Six principles guide Tangle's design. Each addresses a specific failure mode in existing infrastructure.

1.3.1 Democratized Ownership

The dominant model for AI infrastructure concentrates value in cloud companies. They capture margin on every API call and own the customer relationship. Developers and operators become interchangeable suppliers.

Tangle inverts this model. Value flows to those who create it: developers earn from blueprints, operators earn from compute, delegators earn from stake. The protocol captures only a governance-controlled fee (currently 19.5%, with a 0.5% keeper rebate carved out) funding ongoing development. This fee is not rent but payment for coordination.

The inflation distribution allocates 10% to developers and 25% to operators. Blueprint creators share in network growth alongside compute providers. Payment splits route value to participants who earned it. Governance grants voting power proportional to stake.

1.3.2 Permissionless Participation

Permission systems create gatekeepers. Gatekeepers accumulate power. Power corrupts. Every permissioned system eventually serves those who control permissions rather than those who need access.

Tangle implements permissionless participation at every layer. Anyone can become an operator by staking the minimum bond. Anyone can deploy a blueprint. Anyone can use services. No committee approves applications. Participation requires keys and payment rather than identity, so machine customers are first-class: an agent holding a wallet becomes a customer the same way a human does.

Abuse prevention uses economic rather than administrative mechanisms. Minimum stake creates Sybil resistance without identity. Schema validation ensures well-formed services without

human review. Slashing punishes misbehavior without prior permission. Markets filter quality more effectively than committees could.

1.3.3 Developer Expressiveness

Protocols face a fundamental choice: specific functionality or general primitives. Specific functionality limits what developers build. General primitives enable arbitrary applications.

Tangle chooses expressiveness. The hook system enables custom logic at every lifecycle stage: before operator registration, after service activation, during job execution, upon termination. Blueprints define their own pricing models, membership rules, slashing conditions, and verification mechanisms.

This enables use cases the protocol designers did not anticipate. A blueprint for AI inference might implement optimistic execution with fraud proofs. A blueprint for data processing might implement redundant execution with majority voting. The protocol executes hooks blueprints define without understanding the verification mechanisms.

1.3.4 Economic Security

Trust is expensive. Legal agreements require enforcement. Reputation requires history. Authorities require audits. These mechanisms impose costs that scale poorly.

Cryptoeconomic security provides an alternative: trust through stake. Operators deposit assets that can be destroyed if they misbehave. The cost of cheating exceeds any benefit. Trust emerges from aligned incentives, not reputation or authority.

The security condition is explicit: for any attack, the expected cost (detection probability $\times$ slash amount) must exceed expected benefit. Blueprints configure parameters based on threat models. High-value services require higher exposures. The protocol provides the framework; blueprints configure specifics.

1.3.5 Scalable Efficiency

Gas costs determine what is practical on EVM chains. Operations costing millions of gas are impossible. Naive implementations of staking and reward distribution iterate over participants. For a million delegators, this exceeds any block gas limit.

The core accounting paths are O(1). Share-based accounting tracks delegation through exchange rates rather than individual balances. Accumulated-per-share rewards track distributions through single accumulators. Proportional slashing adjusts exchange rates without iterating over positions.

This is a design requirement, not a micro-optimization. A protocol that iterates over every delegator cannot support a large operator market on an EVM chain.

1.3.6 Isolation by Default

AI agents present unique security challenges. Traditional software executes predictable code. AI agents execute actions that emerge from model weights, actions that may surprise creators. An agent told to optimize revenue might discover that competitor unavailability increases revenue.

Isolation by default addresses this uncertainty. Sandboxes enforce explicit permissions. Containers separate processes. Resource limits prevent exhaustion. Logging captures all actions for audit.

The cost of isolation is functionality. Agents that cannot access the network cannot fetch data. Blueprints must explicitly grant capabilities, creating tradeoff between security and functionality. The default is secure; developers opt into risk.

1.3.7 Navigating Tensions

Principles sometimes conflict. Permissionless participation allows low-quality operators. Scalable efficiency requires complexity. Economic security requires locked stake.

Tangle navigates through mechanism design. Permissionless participation combined with market selection allows anyone to enter while quality emerges. Scalable efficiency implemented behind simple interfaces provides both scale and usability. Economic security with flexible exposure lets blueprints configure tradeoffs.

The most significant tension is between expressiveness and security. Hooks enabling arbitrary logic also enable bugs. Tangle provides safe defaults, validates inputs, and isolates failures. Hooks cannot corrupt protocol accounting; they affect only their blueprint's behavior.

2 Blueprints and Services

The blueprint system is Tangle's service architecture. A blueprint defines a service type. A service instance is the live deployment of that blueprint with a selected operator set.

2.1 What Is a Blueprint?

A blueprint is a reusable template for a service type. Developers use it to specify computational tasks, pricing models, operator requirements, and optional custom logic through hooks.

Formally, a blueprint comprises:

- Metadata for discovery (name, description, author)
- Configuration for pricing and membership
- Job definitions specifying tasks operators execute
- Validation schemas ensuring well-formed requests
- Optional custom service manager implementing hooks

Blueprints are identified by unique numeric IDs assigned at creation. Once an operator registers, the protocol locks blueprint metadata, preventing developers from modifying terms after operators have committed.

2.2 What Is a Service Instance?

A service instance is a running deployment of a blueprint with assigned operators. The blueprint says what can be run; the service instance is the thing customers actually use.

A service comprises:

- Reference to its blueprint
- Set of participating operators with committed exposures
- Configuration parameters validated against the schema
- Time-to-live (TTL) defining service duration
- Payment configuration specifying token and amount

Services move through defined states: pending (awaiting operator approvals), active (operational), and terminated (completed lifecycle).

2.3 Service Lifecycle

The lifecycle proceeds through request, approval, activation, operation, and termination.

A customer initiates by submitting a request specifying the blueprint, desired operators, configuration, payment, TTL, permitted callers, and security requirements. The protocol validates parameters and notifies operators.

Each operator must respond. Approval requires committing an exposure (fraction of stake backing the service). If any operator rejects, the service fails. When all approve, the service activates.

Upon activation, operators provide the service. Customers submit jobs; operators execute and submit results. The payment system distributes funds according to the configured model.

Services terminate through customer request, TTL expiration, or failure to maintain minimum operator count. Remaining escrow refunds to the customer.

2.4 Pricing Models

Four payment models accommodate different service types:

Pay-once collects a single upfront payment, distributed when operators approve. This suits one-time computations.

Subscription collects recurring payments from customer-funded escrow. Billing is permissionless and TWAP-fair: anyone may call `billSubscription`, and the 0.5% keeper rebate makes doing so EV-positive without a privileged scheduler. This suits ongoing services.

Event-driven collects payment per job execution. This suits variable workloads.

RFQ quotes let operators sign binding, EIP-712 prices for a specific request, which the customer accepts and executes on-chain (see §4). This suits price discovery across heterogeneous operators.

2.5 Membership Models

Fixed membership locks the operator set at service creation. Operators cannot join or leave once activated.

Dynamic membership permits operators to join after activation and leave subject to exit queue constraints. Long-running services can change operator sets without recreating the blueprint.

2.6 Master Blueprint Service Manager (MBSM)

Blueprints may define custom service managers that implement hooks, code executed at lifecycle events. The Master Blueprint Service Manager (MBSM) provides protocol-wide management of these service managers.

2.6.1 Architecture

Two contracts compose the MBSM system:

MasterBlueprintServiceManager is the protocol-wide sink for blueprint definitions. When blueprints are created, Tangle records the definition here. This provides:

- Permanent record of all blueprint definitions
- Governance inspection and curation capabilities
- Historical audit trail for blueprint metadata

MBSMRegistry manages versioning of MBSM implementations. This enables:

- Protocol-wide upgrades: new MBSM versions apply to all blueprints using "Latest"
- Blueprint pinning: developers can pin to specific versions for stability
- Graceful deprecation: 7-day grace period before old versions become unusable

These three roles are distinct: the `MasterBlueprintServiceManager` is the protocol-side dispatcher that receives lifecycle hook calls and forwards them to the right blueprint manager; the `MBSMRegistry` is the version registry that maps revision numbers to dispatcher implementations; and `BlueprintServiceManagerBase` is the developer-facing base contract a blueprint author extends to implement custom hooks for their own blueprint.

2.6.2 Version Management

The registry tracks MBSM versions by revision number (1-indexed). Blueprints can:

Use Latest: By default, blueprints use the most recent MBSM version. Protocol upgrades automatically apply.

Pin to Version: Developers requiring stability can pin their blueprint to a specific revision. The pinned version persists until explicitly changed.

When governance deprecates a version:

1. Deprecation initiates, starting a 7-day grace period
2. During grace period, the version still functions but emits warnings
3. After grace period, governance can complete deprecation
4. Pinned blueprints should re-pin before completion

Emergency deprecation bypasses the grace period for security incidents.

2.6.3 Why This Matters

MBSM handles a practical upgrade problem: the protocol needs to evolve, but live services need stable rules. A trading-agent service instance cannot have its service manager change mid-operation. Pinning gives that service a fixed manager while the protocol can still ship new MBSM versions.

3 Operators and Staking

Operators are staked entities that run services and earn rewards. The staking system provides economic security backing service guarantees.

3.1 Operator Registration

An entity becomes an operator by depositing a self-bond meeting minimum stake requirements. The bond is single-asset: at least 1,000 TNT, always denominated in TNT (delegation may accept other assets, but the operator self-bond does not). Registration creates metadata tracking self-stake, delegation count, and status. Operators set a commission (default 10%) on delegator rewards; commission changes are subject to a 7-day timelock. Delegations have a floor of 0.2 TNT.

Operators then register for specific blueprints, providing an ECDSA public key for gossip identity (with an optional BLS key for threshold/aggregate signing) and an RPC endpoint for communication. Quotes operators sign use EIP-712. The protocol validates that the blueprint is active, minimum stakes are met, and schema validation passes.

3.2 Delegation Modes

Operators configure delegation mode:

- **Disabled**: Only self-stake accepted
- **Whitelist**: Only approved addresses may delegate
- **Open**: Any address may delegate

3.3 Liquid Delegation Vaults

Tangle implements liquid delegation through `LiquidDelegationVault`, an ERC-7540 compliant vault enabling asynchronous deposit and redemption of delegation positions. Delegators deposit assets and receive vault shares representing their proportional claim on the operator's staked

position. These shares are transferable, enabling liquid delegation positions that can be used in DeFi while the underlying assets remain staked.

The vault uses asynchronous request-fulfill flow per ERC-7540: delegators submit deposit or redemption requests, which are fulfilled after the protocol's delay periods. Time advances in rounds of 6 hours; delegator exits clear after 28 rounds (7 days) and operator exits after 56 rounds (14 days). The vault applies a default 15% operator commission on accrued rewards. A virtual share offset ($V = 10^3$) prevents first-depositor attacks on the share exchange rate. When slashing occurs, total vault assets decrease while shares remain constant, automatically distributing losses proportionally across all vault participants.

3.4 Exposure Selection

When operators approve service requests, they commit an exposure in basis points (0-10000). This determines what fraction of stake backs the service.

Exposure has two consequences: it determines reward share (higher exposure = more rewards) and bounds slashing (operators lose at most their committed percentage).

The protocol's `ExposureManager` implements granular per-asset exposure tracking. Rather than a single exposure value per operator per service, the system maintains exposure limits for each staked asset independently. Operators can configure per-asset exposure caps, and services can specify minimum and maximum exposure requirements per asset type. The `ExposureCalculator` computes effective exposure across an operator's portfolio, enabling precise risk management when operators participate in multiple services with different collateral compositions.

This design enables participation in multiple services with bounded total risk.

3.5 O(1) Share-Based Accounting

The staking system uses share-based accounting for O(1) operations. Each operator maintains a reward pool tracking total shares and total assets.

When delegating amount x to a pool with state (T_s, T_a), the delegator receives shares:

$$h = x \cdot \frac{T_s + V}{T_a + V}$$

where V is a virtual offset (typically 10^3) preventing first-depositor attacks; see Appendix A.1.

The delegation's value is $v = h \cdot (T_a/T_s)$. When assets change through rewards or slashing, this value changes proportionally without per-delegator updates.

When slashing occurs, only total assets decrease. Shares remain constant. The exchange rate decreases, affecting all delegators proportionally. No iteration is required.

4 The Service Marketplace

The service marketplace bridges supply (operators) with demand (customers) through request-for-quote pricing.

4.1 Why Request-for-Quote

Service pricing mechanisms face design choices. Fixed pricing cannot adapt to changing costs. Order books work poorly for heterogeneous services. AMMs struggle with operator differentiation.

Request-for-quote asks operators to provide binding prices for specific requests. Customers compare price, resource mix, exposure, and operator history, then select a quote. RFQ fits heterogeneous operators, requires no standing on-chain order book, and gives final pricing at service creation.

4.2 The RFQ Flow

Operators register with ECDSA and BLS keys and RPC endpoints. They run software that listens for quote requests and returns signed quotes containing:

- Blueprint ID and service duration (TTL)
- Total cost and payment token
- Timestamp and expiry
- Security commitments (per-asset exposure commitments)
- Resource pricing breakdown

Customers collect quotes, verify terms, and submit all quotes in a single transaction. The protocol verifies signatures, checks expiry and replay protection, validates registrations, and confirms payment.

4.3 Resource Pricing

Quotes include granular resource pricing via `ResourcePricing` entries. Each entry specifies a resource kind (CPU, MemoryMB, StorageMB, GPU), a quantity, and a per-unit price in the payment token. This enables customers to compare operators on a per-resource basis rather than opaque flat rates.

Operators competing for the same blueprint quote different resource prices based on their hardware, utilization, and margins. A GPU-heavy AI inference operator prices GPU cycles competitively but charges more for storage; a lightweight oracle operator offers low CPU prices. The market discovers efficient pricing without protocol-imposed schedules.

4.4 Security Commitments and Developer Requirements

Quotes include `AssetSecurityCommitment` entries specifying the exact exposure (in basis points) the operator pledges per staked asset. These commitments are binding: once accepted, the operator's stake is locked at the committed exposure level for the service duration.

Blueprint developers control the security floor through `AssetSecurityRequirement` entries on their blueprint, specifying minimum and maximum exposure per asset type. An AI training blueprint might require at least 50% exposure on staked ETH; a lightweight notification service might cap exposure at 5%. Quotes that fail to meet these requirements are rejected at the protocol level.

This two-sided mechanism gives developers fine-grained control over the economic security backing their service, while operators retain flexibility to differentiate on price and risk appetite within those bounds.

4.5 Quote Security

EIP-712 signatures bind quotes to specific chains and contracts. Each quote includes a unique timestamp; the protocol tracks used quotes. Quotes expire after their specified time. Maximum quote age (default one hour) ensures quotes reflect current conditions.

4.6 MEV Considerations

Quote sniping fails because quotes specify security commitments only registered operators can provide. Price manipulation is mitigated by competition. Sandwich attacks have limited applicability since RFQ transactions do not interact with liquidity pools.

5 Job Execution

Jobs are the fundamental unit of work in Tangle. Understanding how jobs execute, aggregate, and fail is essential for blueprint developers.

5.1 Job Submission

A job is submitted by calling the service with a job index (identifying which task) and ABI-encoded inputs. The protocol validates:

- Caller is authorized (service owner or permitted caller list)
- Service is active
- Job index is valid for the blueprint
- Payment is sufficient (for event-driven pricing)

Upon validation, the protocol assigns a call ID and emits a `JobCalled` event. Operators monitoring the service receive the event and begin processing.

5.2 Result Submission

Operators execute the job according to blueprint logic and submit results. The protocol validates:

- Operator is registered for the service
- Call ID exists and is pending
- Operator has not already submitted for this call

For single-operator jobs, the first valid result completes the job. For aggregated jobs, results accumulate until reaching the required threshold.

5.3 Aggregation Modes

The blueprint's service manager defines aggregation requirements through `getRequiredResultCount`:

Single result (count = 1): First valid submission completes the job. Suitable for speed-critical applications where any operator's result is acceptable.

Multi-result (count > 1): Job waits for the specified number of submissions. Suitable for:

- Verification (2 results): Basic redundancy, detects single-operator failures
- Consensus (3+ results): Byzantine fault tolerance, majority determines truth
- Threshold cryptography: Requires specific participant count for key operations

The service manager can also implement custom aggregation logic in the `onJobResult` hook, comparing results, rejecting outliers, or computing weighted averages.

5.4 Failure Modes

5.4.1 Operator Timeout

An operator receives a job but fails to respond. Causes include network partition, hardware failure, software crash, or deliberate non-response.

Detection: The protocol does not enforce timeouts directly. Blueprints implement timeout logic through keeper services that monitor job age and propose slashes for non-responsive operators.

Recovery: For single-operator jobs, customers may resubmit. For multi-operator jobs, remaining operators may complete the threshold without the failed operator. Services using dynamic membership may replace the failed operator.

Consequences: Non-response typically triggers heartbeat-based slashing if the blueprint implements it.

5.4.2 Result Disagreement

Operators submit conflicting results for the same job. This indicates either operator error (bugs, different data sources) or malicious behavior.

Detection: The `onJobResult` hook receives all submissions and can compare. Simple strategies include exact-match requirements; sophisticated strategies include deviation thresholds or median selection.

Recovery: Blueprint-dependent. Options include accepting the majority result, rejecting the job entirely, or flagging for human review.

Consequences: Outlier operators may be slashed. The blueprint defines what constitutes acceptable deviation.

5.4.3 Invalid Results

An operator submits a result that fails validation (malformed encoding, impossible values, failed cryptographic verification).

Detection: Schema validation at submission time; blueprint hooks can perform semantic validation.

Recovery: Invalid submissions are rejected. The operator may resubmit if the deadline permits.

Consequences: Repeated invalid submissions indicate operator malfunction or malice, triggering investigation or slashing.

5.4.4 Service Unavailability

The entire service becomes unavailable (all operators offline, network partition).

Detection: Heartbeat monitoring detects when operators stop responding. Customers experience submission failures.

Recovery: Service resumes when operators return. Pending jobs may timeout and require resubmission.

Consequences: Prolonged unavailability triggers slashing per the blueprint's SLA requirements.

5.5 Job Execution Economics

Job execution costs include:

- Gas for on-chain submission and result recording
- Off-chain compute costs for job processing
- Coordination overhead for multi-operator aggregation

These costs inform pricing. Operators quote prices covering their costs plus margin. Event-driven pricing passes per-job costs to customers directly.

6 Fee Distribution and Rewards

Value flows through Tangle via service fees and inflation rewards.

6.1 Service Fee Splits

When customers pay, the payment splits across five recipients. Default allocation:

Recipient	Share
Operators	40%
Developer	20%
Stakers (delegators)	20%
Protocol Treasury	19.5%
Keeper rebate	0.5%

These percentages are governance-controlled. The 0.5% keeper rebate is carved from the protocol share and paid to whoever triggers permissionless subscription billing, making `billSubscription` EV-positive for keepers. The staker (delegator) share captures rounding dust.

6.2 Operator Revenue

The operator allocation distributes weighted by committed exposure:

$$\text{operatorShare}_i = \frac{\text{operatorAmount} \times e_i}{\sum_j e_j}$$

Operators who commit more stake earn more from that service.

6.3 Delegator Rewards

Lock duration multipliers reward longer commitments:

Lock Duration	Multiplier
None	1.0x
One Month	1.1x
Two Months	1.2x
Three Months	1.3x
Six Months	1.6x

A delegator's score is amount × multiplier. Rewards distribute proportionally using accumulated-per-share accounting for O(1) efficiency.

6.4 Streaming Payments

For subscription-based services, the `StreamingPaymentManager` distributes payments continuously over the service's time-to-live (TTL) rather than as a lump sum. When a customer funds a service, the payment enters a stream that releases tokens proportionally as time elapses. Operators can claim accrued payments at any point. If a service terminates early, unstreamed funds return to the customer.

This mechanism aligns payment with service delivery: operators earn as they work, customers pay only for time consumed, and early termination automatically handles refunds without manual intervention.

6.5 Developer Rewards

Developers earn from service fees and inflation rewards through scoring:

$$\begin{aligned} \text{blueprintScore} &= \text{blueprintCount} \times 500 \\ \text{serviceScore} &= \text{serviceCount} \times 1000 \\ \text{jobScore} &= \text{jobCount} \times 100 \\ \text{feeScore} &= \sqrt{\text{totalFees}/10^{18}} \times 10^9 \end{aligned}$$

The square root on fees prevents whale dominance.

7 Inflation and Epoch Distribution

Inflation rewards provide income while service demand develops, encouraging early participation.

7.1 Token Supply, Genesis Allocation, and Migration

TNT is hard-capped at `MAX_SUPPLY` = 100,000,000 TNT. The full cap is minted at genesis across four buckets, so `totalSupply` equals the cap from block zero:

Genesis Bucket	Amount (TNT)
Substrate (migration claims)	∼51.24M
EVM (genesis EVM accounts)	∼1.13M
Treasury	∼32.59M
Foundation	∼15.04M

Because the genesis supply already equals the cap, no new TNT can be created in practice. The supply is hard-capped; the InflationPool cannot mint and only distributes TNT it already holds; emission is treasury-funded; and while a governance `MINTER_ROLE` exists, it can mint only up to the cap, so with genesis = cap it has no headroom (`mint()` reverts). What the protocol calls "inflation" is therefore an *emission*: TNT transferred from the Treasury bucket into the InflationPool, not freshly issued supply.

Migration and vesting. Substrate balances migrate to the EVM via claims proven with an SP1 Groth16 zero-knowledge proof of Substrate key ownership plus a Merkle inclusion proof. Claimed allocations vest: 10% is unlocked at claim, followed by a 12-month cliff and 24-month linear release. Treasury and foundation allocations vest over roughly three years, with 30% of the foundation allocation unlocked at the start. Because most of the supply is vesting-locked at launch, the delegatable and votable float is thin, a constraint that shapes the governance parameters below.

7.2 The Pre-Funded Model

The InflationPool uses a pre-funded model. The pool cannot mint tokens; it distributes tokens it holds. Emission is funded by transfers from the Treasury bucket into the pool via `fund()`, targeting roughly 1% of supply per year (∼1,000,000 TNT/yr). These transfers move already-minted TNT; they dilute no holder and create no new supply.

Each epoch's budget:

$$\text{epochBudget} = \frac{\text{poolBalance}}{\text{epochsRemaining}}$$

This smooths distribution over the funding period.

7.3 Distribution Weights

Default allocation:

Category	Weight
Staking (delegators)	55%
Operators (performance)	25%
Customers (usage)	10%
Developers (merit)	10%
Stakers (exposure rail, 0% at genesis)	0%

The protocol defines five distribution categories. The exposure-rail "stakers" category rewards participants who restake assets from external protocols (e.g., beacon chain validators) once live services use that rail. It is set to 0% at genesis because the beacon-chain and L2 restaking paths are deployment-gated for Phase 2 (see §12). Governance can adjust all weights to steer network growth.

7.4 Operator Scoring

Operator inflation rewards use performance metrics:

$$\text{successRate} = \frac{\text{successfulJobs} \times 10000}{\text{totalJobs}}$$

$$\text{stakeWeight} = \frac{\text{totalStake}}{10^9}$$

$$\text{jobScore} = \frac{\text{jobs} \times \text{successRate} \times \text{stakeWeight}}{10000}$$

Stake weight is linear to defeat Sybil attacks.

8 Slashing and Accountability

Slashing provides consequences for misbehavior, making economic security meaningful.

8.1 Developer-Defined Conditions

The protocol provides slashing mechanism; developers define when to use it. Each service type has different requirements the protocol cannot anticipate.

Developers implement conditions through hooks: `querySlashingOrigin` specifies who may propose slashes; `onUnappliedSlash` executes when slashing finalizes.

Combined with the SDK's QoS package (see *Blueprint SDK Guide*, Quality of Service), developers can build automated SLA enforcement. QoS monitors collect operator metrics (heartbeat, latency, availability, throughput). When metrics breach developer-defined thresholds, the monitor proposes a slash through the protocol. The blueprint's `querySlashingOrigin` hook authorizes the monitor, and the dispute window gives operators recourse. This creates a closed loop: developers set quality requirements, operators commit resources, QoS measures compliance, and slashing enforces consequences.

8.2 The Dispute Window

Slash proposals do not execute immediately. A dispute window provides operators opportunity to contest, and the resolution deadline (`disputeResolutionDeadline`, 21 days at launch) is *fail-open*: if a dispute is left unresolved past the deadline, the slash auto-executes. Disputing

requires posting a `disputeBond` of 0.02 ETH, refunded if the slash is cancelled and forfeited if it executes, which prices out griefing disputes.

The lifecycle:

1. **Proposal**: Evidence submitted, pending proposal created. The operator is immediately frozen from unstaking while any slash is pending (anti-evasion)
2. **Dispute**: Operator may post the dispute bond and submit counter-evidence; administrators may cancel
3. **Execution**: After the window expires (or the resolution deadline lapses unresolved), any address may execute undisputed slashes

8.3 Proportional Slashing

Slashing is proportional to committed exposure:

$$\text{effectiveSlashBps} = \frac{\text{slashBps} \times \text{exposureBps}}{10000}$$

An operator with 10% exposure faces at most 10% of stake at risk.

8.4 Withdrawal Protection

Pending slashes block withdrawals. Delegators cannot withdraw while slashes are pending against their operator.

9 Governance

Token holders control protocol evolution through on-chain governance.

9.1 Governance Philosophy

Tangle prioritizes **safety over liveness**. Proposals that fail do less damage than malicious proposals that pass. Thresholds are high, quorum requirements ensure participation, and timelocks provide escape hatches.

Stake-weighted voting gives those with more at stake more say. Economic exposure confers governance rights, not democratic in the one-person-one-vote sense, but democratic in the shareholder sense.

On-chain execution with off-chain deliberation. Formal votes occur on-chain with cryptographic finality. Discussion and consensus-building occur off-chain through forums and calls. The mechanism records decisions; the process generates them.

9.2 Governance Mechanism

On-chain governance uses OpenZeppelin Governor with ERC20Votes on an ERC-6372 timestamp clock (all windows are wall-clock durations, not block counts):

Parameter	Launch value	Rationale
Proposal threshold	100,000 TNT (flat)	∼0.09% of supply; lets a small set of holders propose
Voting delay	1 day	Time for review and position preparation
Voting period	7 days	Defense window after attack detection
Quorum	4% of total supply	Reachable given thin delegatable float at launch
Timelock	4 days (uniform)	Single exit window for all actions, including upgrades

Parameter Selection Rationale. The threshold is a flat 100,000 TNT (∼0.09% of supply) rather than a percentage, so a small set of committed holders can propose. The quorum is set to a low 4% of total supply because most supply is vesting-locked at launch, leaving a thin delegatable and votable float; a percentage-of-supply quorum calibrated to mature DAOs would be unreachable. Quorum is expected to ratchet up as vesting unlocks and float deepens. The real capture defense at launch is not the quorum but a CANCELLER guardian multisig together with the uniform 4-day timelock, which gives affected users a wall-clock window to exit before any passed proposal executes. There is no separate, longer upgrade timelock; upgrades use the same 4-day delay under the highest scrutiny.

9.3 Worked Example: Fee Parameter Change

A proposal to increase the protocol fee share from 19.5% to 22%:

Day 0: A holder (or delegate) controlling at least 100,000 TNT creates a proposal targeting `setProtocolFeeBps(2200)`.

Day 0-1: Voting delay. Community reviews; the snapshot timestamp is recorded for vote-weight accounting. Supporters argue the treasury needs audit funding; operators post objections.

Days 1-8: Voting period (7 days). By day 5, participation passes the 4% quorum with a majority in favor; an opposing coalition raises turnout further as the deadline approaches.

Day 8: Voting closes. Quorum (4% of total supply) is met and a simple majority is in favor, so the proposal succeeds.

Days 8-12: Timelock period (4 days). Operators adjust pricing or begin exit procedures; the canceller guardian can veto a clearly malicious proposal during this window.

Day 12: Anyone calls `execute()`. The protocol fee share updates to 22%.

9.4 Attack Resistance

Governance capture: The 4% quorum is deliberately low because the launch float is thin, so quorum alone is not the defense. The real defenses are the canceller guardian multisig, which can veto a malicious proposal during the timelock, and the uniform 4-day timelock, which gives users a wall-clock window to exit. Quorum ratchets up as vesting unlocks deepen the float. Attacks also trigger defensive participation, and timelocks provide mobilization time.

Flash loan attacks: OpenZeppelin's ERC20Votes uses vote checkpointing on a timestamp clock. Voting power is determined at the proposal's snapshot timestamp, not current holdings. Attackers borrowing tokens after the snapshot gain no power.

Whale manipulation: Large holders have disproportionate influence by design. Those with more at stake should have more say. Whales abusing power face social and economic consequences, their stake's value depends on protocol health.

Voter apathy: Quorum requirements prevent minority rule. Delegation allows passive holders to participate through representatives.

Bribery attacks: Defense relies on long-term incentives. Accepting bribes for harmful proposals damages holders' own stakes.

9.5 Vote Delegation

Token holders may delegate voting power. Delegation is transitive: if Alice delegates to Bob and Bob to Carol, Alice's power accrues to Carol. Delegation can change at any time; current delegation at vote snapshot determines allocation.

Delegation creates a representative layer where informed members accumulate power from less engaged holders. This can improve governance quality or create centralization risks. The community must monitor delegation patterns.

9.6 Emergency Mechanisms

Not all threats wait for standard governance.

Pause functionality: PAUSER_ROLE holders halt protocol operations. Pausing prevents new services, suspends job processing, blocks withdrawals. Unpausing requires governance or ADMIN_ROLE.

Emergency withdrawal: DEFAULT_ADMIN_ROLE can withdraw funds from contracts (e.g., InflationPool) to a designated safe address. Nuclear option for contract bugs threatening fund safety. Logged and auditable.

Guardian multisig: May hold emergency roles during early protocol period. Can act faster than governance in genuine emergencies. Powers revoked or transferred to timelock as protocol matures.

These mechanisms create centralization risk. Mitigation is transparency (all actions on-chain) and accountability (abuse detectable and punishable).

9.7 Governable Parameters

Governance controls comprehensive protocol parameters:

Fee parameters: Operator, developer, staker, protocol, keeper percentages (default 40/20/20/19.5/0.5); TNT payment discounts; minimum fees.

Inflation parameters: Distribution weights (staking 55 / operators 25 / customers 10 / developers 10 / exposure stakers 0); epoch length (7 days); funding period (365 days); minimum stake epochs (1).

Staking parameters: Operator self-bond floor (1,000 TNT, TNT-only); minimum delegation (0.2 TNT); operator commission (default 10%, 7-day timelock); round length (6 hours) and exit delays (delegator 28 rounds, operator 56 rounds); delegation modes; lock multipliers (1.0/1.1/1.2/1.3/1.6x); maximum blueprints per operator.

Slashing parameters: Dispute resolution deadline (21 days, fail-open); dispute bond (0.02 ETH); maximum slash (`maxSlashBps` 50% at launch, governance-raisable per-service for equivocation up to a hardcoded 100% ceiling); instant slash enablement (disabled).

Service parameters: Min/max TTLs; request expiry; maximum quote age (1 hour); heartbeat requirements.

9.8 Upgrade Mechanism

Contract upgrades use UUPS (Universal Upgradeable Proxy Standard). Each upgradeable contract (Tangle, MultiAssetDelegation, InflationPool) deploys behind a proxy. Upgrading replaces implementation while preserving storage.

UUPS places upgrade logic in implementation rather than proxy, reducing proxy size and attack surface. UUPS allows disabling upgradeability if desired. Upgrades follow standard governance with longest timelock (7 days) and highest scrutiny.

9.8.1 Facet Router Architecture

The core `Tangle` contract is a custom facet router behind a UUPS proxy, *not* an EIP-2535 Diamond: there is no `diamondCut` or loupe interface. The protocol's functionality is split across roughly two dozen core mixin facets, each handling a specific domain: blueprints, operators, service lifecycle and approvals, jobs and aggregation, RFQ, payments, slashing, and quotes. The router's `fallback` resolves an incoming function selector to the facet that implements it and `delegatecall`s into it, so all facets execute against the single shared proxy storage, enabling atomic cross-domain operations while keeping individual facet code auditable. Upgrades do not go through a diamond cut; they follow the UUPS path under governance, which can also

update the selector-to-facet routing. Developers interacting with the protocol should be aware that function selectors route through the proxy to the appropriate facet implementation.

Upgrades are the most sensitive governance action. A malicious upgrade could drain funds or disable slashing. Every upgrade proposal should be treated as dangerous until proven otherwise.

10 Security Model

This section synthesizes the individual mechanisms into a coherent security analysis. Security in cryptoeconomic protocols differs from traditional security: the goal is not to make attacks impossible but to make them unprofitable.

10.1 Threat Model

Three adversary classes with distinct capabilities:

Rational adversaries are profit-motivated. They attack if and only if expected value exceeds expected cost. For a rational adversary:

$$\text{Attack iff } V > P_d \cdot S$$

where V is value gained, P_d is detection probability, and S is slash amount. Defense is economic: ensure $P_d \cdot S \geq V$ for all attacks.

Byzantine adversaries may act irrationally, motivated by ideology, sabotage, or external incentives. Defense bounds damage: stake requirements limit participation, exposure limits cap service risk, dispute windows provide response time.

Colluding adversaries coordinate across roles. Operator-customer collusion might generate fake jobs. Operator-developer collusion might design weak verification. Defense requires that verification not depend solely on parties with aligned incentives.

10.2 Economic Security Analysis

10.2.1 Cost of Corruption

The cost of corruption (CoC) is the minimum amount an adversary must control to corrupt a service. For a service with n operators each staking s_i with exposure e_i:

$$\text{StakeAtRisk} = \sum_{i=1}^{n} s_i \cdot e_i$$

If corruption requires controlling k of n operators:

$$\text{CoC} = \min_{\substack{T \subseteq \{1,\ldots,n\} \\ |T|=k}} \sum_{i \in T} s_i \cdot e_i$$

10.2.2 Profit from Corruption

The profit from corruption (PfC) is maximum extractable value. This depends on service type. An oracle has PfC equal to maximum arbitrage profit. A trading agent has PfC equal to assets under management. A data service has PfC equal to data confidentiality value.

Blueprints must analyze PfC and configure exposure accordingly. A service with $1M PfC requires operators staking at least $1M in exposed assets.

10.2.3 Security Ratio

The security ratio is CoC/PfC. Ratios above 1 make honest behavior more profitable under perfect detection.

$$\text{SecurityRatio} = \frac{\text{CoC}}{\text{PfC}}$$

We recommend **minimum ratios of 1.5x**. This accounts for imperfect detection (real $P_d < 100\%$), token price volatility during dispute window, and adversary risk tolerance for high-variance outcomes. High-value services should target 2x or higher.

10.2.4 Worked Example: Trading Agent Service

A trading agent service managing $500,000. Uses 3-of-5 majority voting, adversary must corrupt at least 3 operators.

Operator	Stake	Exposure	Stake at Risk
A	$400,000	50%	$200,000
B	$300,000	60%	$180,000
C	$250,000	80%	$200,000
D	$200,000	100%	$200,000
E	$150,000	100%	$150,000

Cheapest 3-operator coalition is {B, D, E}: CoC = $180K + $200K + $150K = $530K.

PfC = $500K (assets under management).

Security ratio = $530K / $500K = 1.06.

This provides only 6% margin. A 10% token price drop yields ratio 0.95, making attack profitable. If detection probability is 90%, expected cost becomes $0.9 \times \$530K = \$477K$, also making attack marginally profitable.

Recommendation: Increase to at least 1.5x via higher exposures (75-100%), more operators (4-of-7), higher stake requirements ($400K minimum), or monitoring to pause if token prices decline 20%.

10.2.5 Flash Loan Resistance

Flash loans enable acquiring capital without economic exposure. An adversary could borrow tokens, stake, attack, and repay within one transaction.

Tangle prevents this through temporal separation. Stake and delegations cannot be acquired and withdrawn within a single transaction: unbonding clears only after multi-day, round-based exit delays (delegator 28 rounds, ∼7 days; operator 56 rounds, ∼14 days), so a flash loan repaid in the same transaction can never both stake and exit. Slashing proposals also survive their dispute window and the 21-day fail-open resolution deadline, long after any flash loan repayment.

10.3 Attack Vectors and Mitigations

Withdrawal Front-running. An operator learning of impending slash might withdraw stake first. *Mitigation*: Pending slashes block withdrawals. `pendingSlashCount` increments on proposal, decrements on resolution. Withdrawals revert while non-zero.

Stake Manipulation. Share-based accounting can be vulnerable to donation or first-depositor attacks. *Mitigation*: Virtual offset V in share calculation prevents first-depositor extraction. Donation attacks (depositing without minting shares) are benign, donors lose value, existing stakers gain.

Slashing Griefing. Proposing slashes against innocent operators, forcing dispute costs. *Mitigation*: Slash proposal requires governance approval or comes from blueprint's service manager. Blueprints allowing open proposals must implement griefing protections (e.g., slasher bonds forfeited if cancelled).

Service Denial. Requesting services without intent to use, consuming operator capacity. *Mitigation*: Payment accompanies requests. Denial requires paying for it.

Operator Collusion. Operators serving same service collude to provide false results. *Mitigation*: Blueprint-specific. Services requiring BFT use verification detecting inconsistency. Services requiring confidentiality use TEEs. Structural mitigations include stake distribution requirements and randomized assignment.

Governance Capture. Accumulating voting power to pass self-serving proposals. *Mitigation*: High thresholds, long voting periods, timelocks. Governance cannot modify core invariants without UUPS upgrades requiring higher scrutiny.

Oracle Manipulation. Corrupted price feeds enable arbitrage against DeFi protocols. Oracle manipulation affects any system relying on external data. *Mitigations at protocol level*: Tangle's multi-operator model enables median aggregation across independent sources, manipulating 1 of 5 oracle operators produces no profit. Time-weighted averaging resists flash manipulation. *Mitigations at blueprint level*: Blueprints define required operator count, aggregation method (median, TWAP, threshold signatures), and source diversity requirements. High-value oracles require operators staking above PfC. Cross-operator comparison flags deviations for investigation. *Residual risk*: Fundamental data source corruption (e.g., all exchanges manipulated simultaneously) cannot be detected by operator redundancy alone, this requires source-level diversification which blueprints can enforce via attestation of data provenance.

10.4 Formal Security Properties

Property 1 (Withdrawal Safety): An operator with pending slash cannot reduce stake below slash amount until resolution.

Mechanism: `pendingSlashCount` blocks withdrawals when non-zero.

Property 2 (Proportional Slashing): When $x\%$ slash executes, all delegators lose exactly $x\%$ of delegation value.

Mechanism: Share-based accounting adjusts `totalAssets` without modifying `totalShares`. Exchange rate decreases proportionally.

Property 3 (Dust Conservation): Payment splits distribute exactly the input amount.

Mechanism: Final recipient receives remainder after floored amounts: final $=$ total $-$ $\sum$ others.

Property 4 (Temporal Separation): Stake cannot participate until locked for `minStakeEpochs`.

Mechanism: Operator registration records stake epoch. Service approval validates current epoch exceeds registration by minimum.

Property 5 (Bounded Governance): Proposals cannot execute until timelock elapses.

Mechanism: `TangleTimelock` enforces minimum delay. Emergency functions require guardian multisig.

10.5 Security Assumptions

Assumption 1 (Rational Majority): Majority of stake is controlled by rational actors who will not attack when expected value is negative.

Assumption 2 (Cryptographic Hardness): ECDSA, BLS signatures, hash functions, and Merkle proofs remain secure.

Assumption 3 (Contract Correctness): Deployed contracts correctly implement specified behavior. Audits and bug bounties mitigate but do not eliminate risk.

Assumption 4 (Economic Stability): Token prices remain stable enough that slashing costs maintain deterrent value.

10.6 Security Recommendations

1. **Size stake appropriately**: Minimum stake should exceed maximum corruption profit
2. **Implement detection**: Slashing is effective only if misbehavior is detected
3. **Use multi-operator services**: Single operators are single points of failure
4. **Monitor conditions**: Pause if token price declines reduce security margins
5. **Participate in governance**: Governance capture is systemic risk

10.7 Residual Risks

Cryptoeconomic security is not absolute security. Several risks remain:

Detection failure. Slashing deters only detected misbehavior. Sophisticated attacks may evade detection entirely. Blueprint designers must analyze detection probability per attack vector and configure slashing proportionally.

Token volatility. Security margins assume stable token prices. Rapid price declines can make attacks profitable before services pause. Circuit breakers and diversified collateral partially mitigate but do not eliminate this risk.

Smart contract bugs. The protocol's security guarantees depend on correct contract implementation. While audits and bug bounties reduce risk, they cannot eliminate it. Upgradability enables fixes but introduces governance risk.

Coordination failures. Multi-operator services assume independent operators. Actual independence is difficult to verify, operators may share infrastructure, ownership, or incentives. Structural mitigations (geographic distribution, stake source diversity) help but cannot guarantee independence.

These residual risks are inherent to cryptoeconomic systems. Tangle provides tools for managing them, the protocol cannot eliminate them.

11 Comparison to Existing Protocols

11.1 The Restaking Landscape

Restaking protocols share a premise: staked assets can secure additional systems beyond their primary use. Each approaches this differently.

EigenLayer pioneered restaking for ETH validators, enabling them to opt into additional slashing conditions for Actively Validated Services (AVSs). Architecture: operators register with AVSs, which define slashing conditions. Slashing proposed by AVS committees, executed through veto-able process. Primary use case: extending Ethereum's validator set to secure bridges, oracles, and rollups.

Symbiotic accepts diverse collateral types (ETH, stablecoins, LSTs) through modular design. Architecture: vaults hold collateral, networks define slashing conditions, resolvers arbitrate disputes. Primary use case: flexible collateral for networks that want assets beyond ETH.

Karak emphasizes multi-chain operation with risk-adjusted collateral matching. Architecture: operates across Ethereum, Arbitrum, and other chains with restaked security. Primary use case: cross-chain applications needing security presence on multiple chains.

11.2 Architectural Comparison

Feature	Tangle	EigenLayer	Symbiotic	Karak
Primary focus	Compute services	Validation	Flexible collateral	Multi-chain
Service definition	Blueprints (code)	AVS contracts	Network configs	DSS configs
Collateral types	Multi-asset	ETH/LSTs	Multi-asset	Multi-asset
Slashing model	Per-service exposure	Binary opt-in	Per-network	Per-DSS
Pricing mechanism	RFQ marketplace	Fixed/market	Fixed/market	Fixed/market
Scaling approach	O(1) algorithms	Varies	Varies	Varies
Job execution	Built-in	External	External	External
Developer rewards	Protocol inflation	None	None	None

11.3 Key Differentiators

Compute-native design. Restaking protocols treat computation as external, AVS operators run whatever software the AVS requires. Tangle treats computation as native: the SDK provides job execution, result aggregation, and P2P coordination. This reduces friction for service developers and operators alike.

Blueprint abstraction. EigenLayer requires deploying new AVS contracts for each service type. Tangle defines service types through blueprints: same protocol contracts, different parameters and hooks. A developer creates a price oracle, an AI inference service, and a keeper bot using identical tooling.

Granular exposure control. Most restaking protocols offer binary participation: opt in or out per service. Tangle's exposure model lets operators tune risk, for example by joining a higher-risk service with 20% exposure while keeping the rest of their stake available. Operators do not have to make all-or-nothing risk decisions.

O(1) scalability. Share-based accounting and accumulated-per-share distributions enable operations that don't iterate over participants. As delegation grows to thousands of operators and millions of delegators, gas costs remain constant. Competitors using per-account iteration face gas limits that constrain growth.

Integrated marketplace. The RFQ system prices heterogeneous services naturally. An AI inference operator with GPUs quotes differently than a lightweight oracle operator. Competitors using fixed pricing or external markets lack this integration.

Developer economics. Tangle allocates 10% of treasury-funded emission to blueprint developers alongside 25% to operators. Service creators get a direct reward path instead of relying only on optional fees.

11.4 When to Choose Each Protocol

Choose EigenLayer when: you need ETH-native security for validation tasks, your service fits the AVS model, and you're building for Ethereum-centric users.

Choose Symbiotic when: you need flexible collateral acceptance, your network benefits from diverse asset types, or you require custom resolver mechanisms.

Choose Karak when: your application spans multiple chains and needs unified security across them.

Choose Tangle when: you're building compute services (AI, data processing, automation), you want SDK tooling for rapid development, you need granular exposure control, or you're building at scale requiring O(1) operations.

12 Beacon Chain Integration

Status: code-complete, deployment gated for Phase 2, not active at mainnet launch. The contracts described in this section are implemented and tested but deploy with `deploy=false` at launch; they are why the exposure-rail "stakers" inflation bucket is 0% at genesis (§7).

Tangle includes a beacon chain module enabling Ethereum validators to restake their beacon chain credentials. The `ValidatorPodManager` manages `ValidatorPod` contracts that verify beacon chain proofs (withdrawal credentials, balance snapshots, validator status), using SP1 and Merkle proofs, to attest restaked ETH positions. Validators deploy a pod, point their withdrawal credentials to it, and gain restaking capacity proportional to their beacon chain balance.

For cross-layer slashing enforcement, the L2 path uses the OP-Stack native cross-domain messenger on Base: the `L2SlashingConnector` and `L2SlashingReceiver` bridge slashing decisions between L1 and L2, authenticated via `xDomainMessageSender`, and a `TangleL2Slasher` executes slashing on L2 based on L1-originated messages. This architecture enables ETH validator restaking on Tangle regardless of which layer the services operate on.

13 Conclusion

Tangle provides the coordination layer for a decentralized compute market. Developers define computational services. Operators provide those services with economic accountability. Customers consume them with cryptographic guarantees.

The mechanisms described here, blueprints and services, staking and delegation, RFQ pricing, fee distribution, slashing, and governance, are designed around one rule: honest participation should be more profitable than cheating. Decentralization is a launch trajectory, not a launch-day fact. At mainnet the UUPS upgrade key sits behind the 4-day governance timelock and a guardian multisig, and most supply is still vesting-locked. As vesting unlocks deepen the delegatable float and governance quorum ratchets up, control diffuses toward operators, developers, and delegators.

The practical argument is straightforward. Decentralized infrastructure reduces provider concentration. Competitive markets pressure pricing. Cryptographic verification gives customers evidence rather than promises. The same properties that made decentralized finance useful apply to decentralized AI infrastructure.

Autonomous work needs an accountability layer. Tangle is built to be that layer.

A Mathematical Foundations

This appendix provides formal definitions for the mathematical constructs referenced throughout the protocol specification.

A.1 Share-Based Accounting

Share-based accounting enables O(1) operations for deposits, withdrawals, and value changes across unlimited participants.

A.1.1 Deposit

When delegating amount x to a pool with total shares T_s and total assets T_a:

$$\text{shares} = x \cdot \frac{T_s + V}{T_a + V}$$

where V is a virtual offset (typically 10^3) preventing first-depositor manipulation.

A.1.2 Withdrawal

When withdrawing h shares:

$$\text{assets} = h \cdot \frac{T_a}{T_s}$$

A.1.3 Value Calculation

A delegation of h shares has current value:

$$v = h \cdot \frac{T_a}{T_s}$$

When pool assets change (rewards or slashing), this value changes proportionally without per-delegator updates.

A.2 Accumulated-Per-Share Rewards

Efficient reward distribution without iterating over recipients.

When rewards R arrive for a pool with total score T:

$$\text{accumulatedPerShare} \mathrel{+}= \frac{R \times 10^{18}}{T}$$

A delegator with score s claims:

$$\text{claimable} = \frac{s \times (\text{accumulated} - \text{lastDebt})}{10^{18}}$$

The precision factor 10^{18} prevents dust loss from integer division.

A.3 Fee Split Calculation

When distributing payment P across recipients with basis point allocations:

$$\begin{aligned}
\text{operatorAmount} &= \lfloor P \times \text{operatorBps}/10000 \rfloor \\
\text{developerAmount} &= \lfloor P \times \text{developerBps}/10000 \rfloor \\
\text{protocolAmount} &= \lfloor P \times \text{protocolBps}/10000 \rfloor \\
\text{keeperAmount} &= \lfloor P \times \text{keeperBps}/10000 \rfloor \\
\text{delegatorAmount} &= P - \text{operator} - \text{developer} - \text{protocol} - \text{keeper}
\end{aligned}$$

The delegator (staker) amount captures rounding dust, ensuring exact distribution.

A.4 Operator Scoring

Operator inflation rewards use performance-weighted scoring:

$$\begin{aligned}
\text{successRate} &= \frac{\text{successfulJobs} \times 10000}{\max(\text{totalJobs}, 1)} \\
\text{stakeWeight} &= \frac{\text{totalStake}}{10^9} \\
\text{jobScore} &= \frac{\text{jobs} \times \text{successRate} \times \text{stakeWeight}}{10000} \\
\text{heartbeatBonus} &= \frac{\text{heartbeats} \times \text{stakeWeight}}{100}
\end{aligned}$$

Linear stake weight (not square root) defeats Sybil attacks, splitting stake across operators provides no scoring advantage.

A.5 Developer Scoring

Developer scoring for inflation rewards:

$$\begin{aligned} \text{blueprintScore} &= \text{blueprintCount} \times 500 \\ \text{serviceScore} &= \text{serviceCount} \times 1000 \\ \text{jobScore} &= \text{jobCount} \times 100 \\ \text{feeScore} &= \sqrt{\text{totalFees}/10^{18}} \times 10^9 \end{aligned}$$

Square root on fees prevents whale dominance. A developer generating $1M scores same as ten developers each generating $10K.

A.6 Proportional Slashing

When slashing operator with exposure e (basis points) by slash severity s (basis points):

$$\text{effectiveSlashBps} = \frac{s \times e}{10000}$$

An operator with 10% (1000 bps) exposure to a service faces at most 10% of stake at risk regardless of slash severity.

A.7 Security Economics

For a service with operators $\{1, ..., n\}$ each with stake s_i and exposure e_i:

$$\text{TotalStakeAtRisk} = \sum_{i=1}^{n} s_i \cdot e_i$$

For k-of-n corruption threshold:

$$\text{CoC} = \min_{\substack{T \subseteq \{1,...,n\} \\ |T|=k}} \sum_{i \in T} s_i \cdot e_i$$

Security condition for rational adversaries with detection probability P_d:

$$P_d \cdot \text{CoC} \geq \text{PfC}$$

Recommended security ratio: $\text{CoC}/\text{PfC} \geq 1.5$.